

## Práctica 4: Automatización de pruebas en Android

---

### 1. Introducción

Los objetivos de esta práctica son los siguientes:

1. Completar el conocimiento que ya poseéis del entorno de desarrollo Android con lo relativo a la realización de pruebas en el mismo.
2. Conocer las herramientas que proporciona Android para realizar pruebas de unidad.
3. Realizar pruebas de unidad sobre la aplicación desarrollada en las practicas de la asignatura de Ingeniería del *Software* y como trabajo de Verificación y Validación.

#### 1.1. Descripción del marco de pruebas de Android

Desde Android Studio, es posible configurar proyectos que realicen pruebas de unidad, de integración y de sistema de aplicaciones Android, tanto en dispositivos reales como en un emulador. El entorno de desarrollo de Android incluye un conjunto de herramientas para realizar pruebas de aplicaciones Android [1]. Algunas de estas herramientas son las siguientes:

**AndroidJUnitRunner** es un ejecutor (*test runner*) para Android compatible con JUnit 4. Android todavía no da soporte a JUnit 5. Pese a que existen *plugins* de Gradle para poder utilizar JUnit 5 en Android,<sup>1</sup> nos vamos a limitar a trabajar con el entorno predeterminado de Android y en esta práctica utilizaremos JUnit 4.

**Espresso** es un entorno de pruebas para interfaces gráficas, adecuado para realizar pruebas de sistema con interfaz gráfica relativas a una aplicación.

---

<sup>1</sup>Como android-junit5 (<https://github.com/mannodermaus/android-junit5>)



## Práctica 4: Automatización de pruebas en Android

---

**UI Automator** es también un entorno de pruebas con interfaces gráficas, adecuado para realizar pruebas de sistema entre aplicaciones, incluyendo el propio sistema Android y las aplicaciones que pueda tener instaladas.

**Exerciser Monkey** es una herramienta para pruebas de estrés que envía secuencias de eventos pseudoaleatorios (clics, toques, gestos y eventos de sistema) a la interfaz de la aplicación que se desea probar, tanto en el emulador como en un dispositivo real.

En esta práctica nos vamos a centrar exclusivamente en la herramienta para realizar pruebas de unidad o pruebas simples de integración: el ejecutor **AndroidJUnitRunner** de JUnit. En la práctica 6.<sup>a</sup> («Automatización de las pruebas para aplicaciones con GUI»), se trabajará con Espresso, UI Automator y Exerciser Monkey.

### 1.1.1. Configuración básica para *testing* en Android

Antes de comenzar con la configuración, comprobad que la versión de la aplicación que tenéis en vuestro repositorio de GitHub compila y funciona correctamente. Una vez que hayáis hecho esta comprobación, podéis aceptar la sugerencia de actualización del *plugin* de Gradle (si os aparece). Comprobad de nuevo que la aplicación funciona y haced un `commit`.

Id a `File >> Project Structure... >> Suggestions` y si hay versiones más recientes de las dependencias de la aplicación, actualizadlas. Comprobad de nuevo que la aplicación funciona y haced un `commit`.

Antes de continuar con la configuración, cread una rama denominada `pr4` y moveos a ella.

En el **fichero «`gradle.properties`»** del proyecto, hay que activar la característica `android.injected.androidTest.leaveApksInstalledAfterRun` para que el complemento de Gradle para Android no desinstale la aplicación después de realizar pruebas, añadiendo la siguiente declaración:

```
android.injected.androidTest.leaveApksInstalledAfterRun=true
```

## 1.2. Pruebas en Android

En Android Studio se pueden ejecutar tanto pruebas de unidad tanto *locales* como *instrumentadas* en dispositivos reales o virtuales.

Las pruebas de unidad locales son pruebas que pueden ejecutarse en la máquina virtual de Java de un equipo de desarrollo sin necesidad de utilizar ni el entorno Android ni

## Práctica 4: Automatización de pruebas en Android

---

ningún dispositivo real ni virtual. Para ello, es necesario que las clases objeto de pruebas no dependan del entorno Android.

Las pruebas instrumentadas son pruebas que se ejecutan en un dispositivo Android real o virtual. Estas pruebas tienen acceso a información de instrumentación como el objeto `Context` de la aplicación que se está probando y otros elementos de la aplicación que dependan de él, como actividades y bases de datos SQLite. Las pruebas instrumentadas pueden utilizarse para pruebas de unidad, de integración o de sistema.

Las pruebas locales son de una ejecución mucho más rápida que las instrumentadas. Es por ello que una buena práctica de diseño en Android consiste en maximizar la cantidad de código que figura en clases que no dependan de la API de Android, mientras que las *actividades* (dependientes de la API) se encargan solo de la interfaz de usuario y los *proveedores de contenidos* (que también pueden depender de la API) de adaptar el acceso a los datos.

### 1.2.1. Pruebas de unidad locales

Las pruebas locales se pueden ejecutar en Android Studio con cualquier entorno de pruebas que esté disponible en IntelliJ.

Entre las actividades a realizar en la práctica, no hay pruebas de unidad locales (van a ser todas instrumentadas, ver sección 1.2.2), pero en el trabajo podría darse el caso de que este tipo de pruebas os fuesen de utilidad.

El código fuente de las pruebas debe ubicarse en un directorio específico («`src/test/java`»), que suele estar creado por defecto. En él os podéis encontrar un fichero denominado «`ExampleUnitTest.java`», que contiene un pequeño test trivial escrito en JUnit 4 que sirve para comprobar si la infraestructura y configuración para ejecutar pruebas de unidad es correcta. Podéis ejecutarlo para comprobarlo.

Es posible utilizar en estos test de unidad locales JUnit 5. Para ello, habría que añadir las siguiente dependencia al fichero «`build.gradle`» del módulo correspondiente a la **aplicación**:

```
dependencies {  
    ...  
    testImplementation "org.junit.jupiter:junit-jupiter:$rootProject.junit5Version"  
    ...  
}
```

En esa dependencia, el valor de la versión de `$rootProject.junit5Version` se resolvería añadiendo la variable `junit5Version` al fichero «`build.gradle`» del **proyecto**. El valor más reciente para ella en el momento de escribir este enunciado es **'6.0.3'**.

Para ejecutar pruebas de unidad locales en Android, en la ventana del proyecto, hay

## Práctica 4: Automatización de pruebas en Android

---

que sincronizar el proyecto y navegar hasta las pruebas de unidad (clases o métodos) y ejecutarlas con el botón derecho del ratón. Para ejecutar todos los test de un directorio, se selecciona el directorio con el botón derecho del ratón y se elige el botón de ejecutar.

### 1.2.2. Pruebas de unidad instrumentadas

Las pruebas de unidad instrumentadas en Android también se escriben a través de pruebas JUnit 4 agrupadas en una o varias clases Java. El código fuente de estas pruebas instrumentadas debe estar ubicado en el directorio «src/androidTest/java», que se crea automáticamente con cada proyecto Android.

En este directorio os podéis encontrar otro test de ejemplo, en un fichero denominado «ExampleInstrumentedTest.java», que contiene un pequeño test trivial escrito en JUnit 4 que sirve para comprobar si la infraestructura y configuración para ejecutar pruebas de unidad es correcta. En particular, este test, que se ejecuta en un dispositivo Android, lanza la aplicación y comprueba el nombre del paquete en el que se ha definido. Podéis ejecutarlo para comprobar que se ejecuta correctamente.

Para realizar pruebas que involucren a una actividad Android, se utiliza en los test la clase genérica `ActivityScenario`,<sup>2</sup> que proporciona una interfaz para lanzar y acceder a actividades Android de una clase concreta desde los test. El entorno de Android también define la clase `ActivityScenarioRule`,<sup>3</sup> que facilita la gestión de los `ActivityScenario`, creándolos antes de cada test y destruyéndolos de forma ordenada al terminar.

De esta forma, si en un test disponemos de un objeto `scenarioRule` de la clase `ActivityScenarioRule<UnaActividadConcreta>`, el cuerpo de la expresión lambda definida por las invocaciones `scenarioRule.getScenario().onActivity(activity -> {...})` se ejecutará en un entorno en el que una actividad de Android de tipo `UnaActividadConcreta` estará correctamente creada e inicializada y accesible a través del parámetro `activity`. En la sección 2 se proporcionan más detalles.

## 2. Actividades a realizar en la práctica

En esta práctica trabajaréis con vuestra aplicación correspondiente al trabajo de esta asignatura, reestructurando el código que escribisteis en la práctica 6.<sup>a</sup> de la asignatura de Ingeniería del *Software* para hacer pruebas basadas en la especificación.

Antes de comenzar a escribir la prueba instrumentada, el código fuente debe ser configurado tal y como se ha explicado en la sección anterior.

---

<sup>2</sup><https://developer.android.com/reference/androidx/test/core/app/ActivityScenario>

<sup>3</sup><https://developer.android.com/reference/androidx/test/ext/junit/rules/ActivityScenarioRule>

## Práctica 4: Automatización de pruebas en Android

---

1. Vamos a crear una primera prueba algo más compleja que la que podemos encontrar en «`ExampleInstrumentedTest.java`» y que también va a ser una *prueba de humo* (una prueba que, si no se pueda ejecutar o falla, representa un síntoma claro de un error de configuración o de código, pero que, en el caso de pasar sin errores, da muy pocas garantías sobre la ausencia de defectos en la aplicación). Esta prueba va a trabajar con el repositorio de *quads* a través de una actividad de vuestra aplicación

Cread una clase de pruebas JUnit. Dado que estamos haciendo pruebas instrumentadas, aseguraos de crear la clase en el directorio «`androidTest`».

2. Añadid a la clase que acabáis de crear la etiqueta `@RunWith(AndroidJUnit4.class)`. No es estrictamente necesario hacerlo, aunque la documentación de Android indica que sí que lo es.
3. Añadid a la clase un atributo público de la clase `ActivityScenarioRule` etiquetado con la etiqueta `@Rule`, de la siguiente forma:

```
@Rule
public ActivityScenarioRule<Notepad> scenarioRule
    = new ActivityScenarioRule<>(Notepad.class);
```

donde tendréis que sustituir `Notepad` por el nombre correspondiente a la clase de vuestra actividad principal o de una actividad a través de la cual podáis tener acceso a vuestro repositorio de *quads*.

Esta declaración crea un objeto (`scenarioRule`) que, en los test, permitirá acceder a una instancia de la clase correspondiente a vuestra actividad. Este atributo, al ser declarado como una regla (etiqueta `@Rule`), creará, a través Android, el objeto de la clase correspondiente antes de que se ejecute cualquier test o método etiquetado con `@Before` y lo destruirá cuando se ejecuten los métodos etiquetados con `@After` tras la ejecución del último test.

4. Añadid una primera prueba que averigüe el número de *quads* que hay en la base de datos de la aplicación, inserte un nuevo *quad* y compruebe que el nuevo número de *quads* es una unidad mayor que el anterior. Para poder hacer esto es necesario acceder al repositorio de *quads* a través de la actividad para la que habéis declarado el objeto `scenarioRule` en el punto anterior, de una forma similar a esta:

```
@Test
public void testCreationIncreasesNumberOfNotes() {
    scenarioRule.getScenario().onActivity(activity -> {
        // Acceso al repositorio a través de la actividad
        ... activity.getNoteRepository() ...
        ...
    });
}
```

## Práctica 4: Automatización de pruebas en Android

---

En el fragmento de código del ejemplo anterior, el acceso al repositorio de notas se hace a través de un método de acceso específico (el *getter* `getNoteRepository()`) añadido a la clase `Notepad` específicamente para poder realizar este test. Probablemente, tendréis que añadir un método similar en vuestra actividad para acceder a vuestro repositorio de *quads*.

Para averiguar el número de *quads*, también tendréis que añadir un método al repositorio de *quads* y otro método *query* a vuestro DAO de *quads*.<sup>4</sup>

Para añadir el *quad* nuevo, podéis utilizar el método de inserción de vuestro repositorio de *quads*.

Recordad que el método que contiene la prueba tiene que estar etiquetado con la etiqueta `@Test`. Por compatibilidad con JUnit 3, el nombre del método que contiene la prueba puede comenzar con el prefijo «test».

5. Añadid un método de finalización (etiquetado, por tanto, con la etiqueta `@After`) que se encargue de borrar el *quad* que se ha creado cuando se ejecuta la prueba solicitada en el apartado anterior. Por compatibilidad con JUnit 3, podéis denominar a este método `tearDown()`.
6. Ejecutad esta primera prueba.

Observad los resultados de la prueba y corregid lo que sea necesario hasta que el resultado sea correcto.

7. Ya hemos comentado que la prueba que habéis programado en el apartado 4, más que comprobar el funcionamiento del repositorio de *quads*, ha servido para comprobar que la configuración del proyecto ha sido correcta y por eso la hemos denominado *prueba de humo*.

Añadid, en la misma clase, un método de prueba **adicional** que realice una prueba algo más exhaustiva: añadid otro *quad* a través del método de inserción del repositorio de *quads*, comprobad que la podéis recuperar a través del identificador generado en la inserción y que los campos del *quad* recuperado son los mismos que los que se especificaron en el momento de la invocación al método de inserción.

Esta prueba sigue sin ser totalmente exhaustiva, pero sí más que simplemente comprobar que el número de *quads* se ha incrementado en 1. Una prueba completamente exhaustiva exigiría asegurarse, además, de que los valores del resto de la base de datos no se han modificado tras la inserción. Por comodidad, y mientras no detectemos errores en las inserciones en la base de datos, nos podemos conformar con la prueba realizada en este apartado.

8. Reestructurad las pruebas que hicisteis en la práctica 6.<sup>a</sup> de la asignatura de

---

<sup>4</sup>En <https://developer.android.com/training/data-storage/room/accessing-data#query> se puede encontrar documentación para realizar una *query* en un DAO.

## Práctica 4: Automatización de pruebas en Android

---

Ingeniería del *Software*, adaptándolas al marco de JUnit y Android. Puede ser buena idea repartir las pruebas en varias clases: una para los casos de prueba de creación de *quads*, otra para los de borrado, una tercera para los de actualización, otras tantas para la creación, borrado y actualización de reservas, una séptima para la prueba de volumen y una octava para la de sobrecarga.

Al finalizar, podréis eliminar las opciones de menú relativas a pruebas de vuestra aplicación, para que ya no le aparezcan al usuario.

### 3. Resultados de la práctica

Esta práctica está directamente vinculada al trabajo de la asignatura y está pensada para ser mejor aprovechada si se realiza en equipo, conforme a los equipos del trabajo de la asignatura.

El código resultado de esta práctica deberá subirse a GitHub en una rama denominada **pr4**. Dicha rama deberá combinarse con la rama **main** una vez se hayan concluido todas las tareas solicitadas en la sección 2 de esta práctica (configuración, pruebas y código corregido). En cualquier caso, esta rama **pr4** no debe borrarse hasta que la práctica haya sido corregida por el profesor.

El plazo de realización de la práctica finaliza el día anterior a la siguiente sesión de prácticas. A partir de esa fecha, el profesor evaluará su resultado a través del último **commit** existente en la rama **pr4**, independientemente de que se haya combinado con la rama **master** o no. Evidentemente, el proyecto correspondiente a dicho **commit** debe poder compilarse y ejecutarse sin errores.

**Los errores y defectos que se encuentren en el código de la aplicación como resultado de la ejecución de estas pruebas se incluirán en GitHub y en el informe de pruebas final del trabajo de la asignatura.**

### 4. Referencias

1. Google. «Test apps on Android». *Android Developers*. <https://developer.android.com/training/testing/>
2. Google. «Test your app». *Android Developers*. <https://developer.android.com/studio/test/>
3. Lars Vogel. «Developing Android unit and instrumentation tests - Tutorial». *Vogella*. <https://www.vogella.com/tutorials/AndroidTesting/article.html>